# Advanced Array Techniques

Interview Prep Revision Notes

Maximum Product Subarray to Difference of Two Arrays

Note: the source notes had no content for question 26 (jumps from 25 to 27). Problems are renumbered sequentially here (21–29) so the set runs without a gap; no problem content was altered.

## Contents

# Q21 Maximum Product Subarray

## INPUT

```
Integer array nums
```

## OUTPUT

Maximum product of any contiguous subarray.

## EXAMPLE

Input: [2,3,-2,4]

Output: 6

```
Because:
[2,3] → 2 × 3 = 6
```

## BRUTE FORCE

Har possible contiguous subarray ka product calculate karo.

```
1. `i = 0 → n-1`
2. `j = i → n-1`
3. `currProd` maintain karo.
4. `maxProd` update karo.
```

Complexity:

Time: $O(n^2)$ Space: O(1)

## OPTIMIZED APPROACH

## PREFIX + SUFFIX PRODUCT

Negative numbers ki wajah se product ka sign change ho sakta hai.

Isliye:

- `pre` → left to right product
- `suff` → right to left product
- `ans` → maximum product

## INITIAL

```
pre = 1
suff = 1
ans = Integer.MIN_VALUE
```

## MAIN LOGIC

```
pre *= nums[i];
suff *= nums[n-i-1];

ans = Math.max(ans, Math.max(pre, suff));
```

Agar product `0` ho jaye:

```
if (pre == 0) pre = 1;
if (suff == 0) suff = 1;
```

WHY RESET TO 1?

Zero ke baad agar 0 se multiply karte rahenge, toh next products bhi 0 ho jayenge.

`1` se reset karne par naya subarray start ho sakta hai.

## WHY `N-I-1`?

```
`nums[i]` → left to right

`nums[n-i-1]` → right to left
```

Example:

```
[2, 3, -2, 4]

i = 0 → nums[3] → 4
i = 1 → nums[2] → -2
i = 2 → nums[1] → 3
i = 3 → nums[0] → 2
```

## DRY RUN

Input:

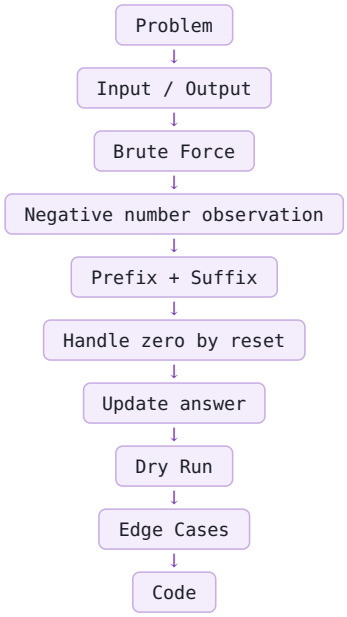
```
[2,3,-2,4]
```

| i | nums[i] | nums[n-i-1] | pre | suff | ans |
|---|---|---|---|---|---|
| 0 | 2 | 4 | 2 | 4 | 4 |
| 1 | 3 | -2 | 6 | -8 | 6 |
| 2 | -2 | 3 | -12 | -24 | 6 |
| 3 | 4 | 2 | -48 | -48 | 6 |

## FINAL ANSWER

```
6
```

## WORKFLOW




## EDGE CASES

```
[2,3,-2,4] → 6
[-2,3,-4] → 24
[-2,0,-1] → 0
[0,2] → 2
[-2] → -2
[0,0] → 0
[2,3,4] → 24
```

## COMPLEXITY

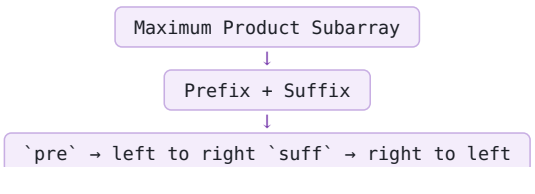
Time: O(n)

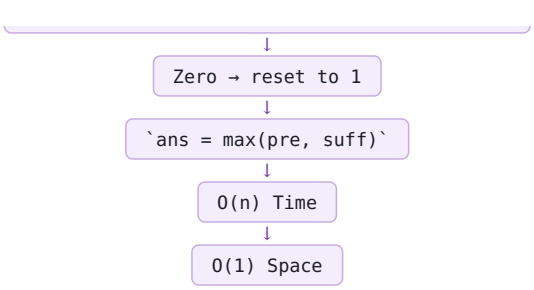
Space: O(1)

## COMPLETE JAVA TEMPLATE

```java
class Solution {

    public int maxProduct(int[] nums) {

        int n = nums.length;

        int pre = 1;
        int suff = 1;
        int ans = Integer.MIN_VALUE;

        for (int i = 0; i < n; i++) {

            if (pre == 0) pre = 1;
            if (suff == 0) suff = 1;

            pre = pre * nums[i];
            suff = suff * nums[n - i - 1];

            ans = Math.max(ans, Math.max(pre, suff));
        }

        return ans;
    }

    public static void main(String[] args) {

        int[] nums = {2, 3, -2, 4};

        // TODO:
        // Apna solution/test code yahan complete karo.

    }
}
```

## CORE LOGIC






## COMMON MISTAKES

```
1. `pre` / `suff` ko 0 se initialize karna.
   → Correct: `1`
```

2. Zero ke baad reset na karna.

```
3. `nums[i]` aur `pre` ko same samajhna.
   → `nums[i]` actual element hai.
   → `pre` cumulative product hai.

4. Sirf prefix use karna.
   → Negative numbers ki wajah se suffix bhi important hai.

5. `n-i-1` bhoolna.
   → Ye right-to-left index deta hai.
```

# Q22 Majority Element II

## INPUT

```
Integer array nums
```

## OUTPUT

All elements whose frequency is more than n/3.

## EXAMPLE

Input: [3,2,3] Output: [3]

## BRUTE FORCE

Har element ki frequency count karo. Agar frequency > n/3 hai to answer mein add karo.
Time: $O(n^2)$ Space: O(1)

> **IMPORTANT OBSERVATION**
> Maximum 2 elements hi n/3 se zyada appear kar sakte hain.

## OPTIMIZED APPROACH

Boyer-Moore Voting Algorithm

```
n/3 case mein maximum 2 candidates ho sakte hain.
```

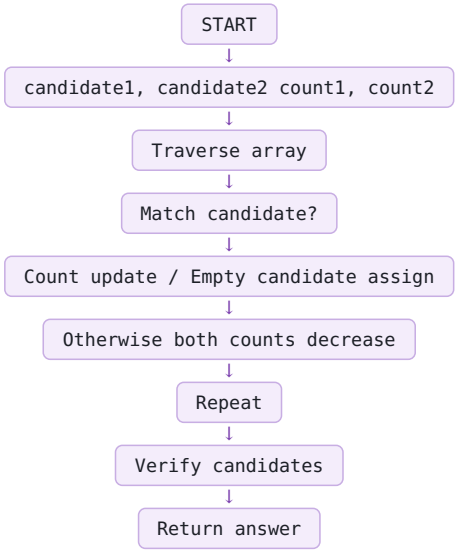
Maintain: candidate1, count1 candidate2, count2

## MAIN LOGIC

```
1. num == candidate1 → count1++
2. num == candidate2 → count2++
3. count1 == 0 → candidate1 = num, count1 = 1
4. count2 == 0 → candidate2 = num, count2 = 1
5. Otherwise → count1--, count2--
```

> **IMPORTANT**
> End mein candidates guaranteed answer nahi hote. Second pass mein actual frequency verify karni hoti hai.
> Agar: frequency > n/3 to answer mein add karo.

## WORKFLOW




## DRY RUN

Input: [3,2,3]

```
Initial:
candidate1 = 0
candidate2 = 0
count1 = 0
count2 = 0

3:
candidate1 = 3
count1 = 1

2:
candidate2 = 2
count2 = 1

3:
count1 = 2
```

Final candidates: 3 and 2

```
Frequency:
3 → 2
2 → 1

n/3 = 1

3 → 2 > 1 → YES
2 → 1 > 1 → NO
```

Answer: [3]

## EDGE CASES

```
[1] → [1]
[1,2] → [1,2]
[1,2,3] → []
[3,2,3] → [3]
```

## COMPLETE JAVA CODE

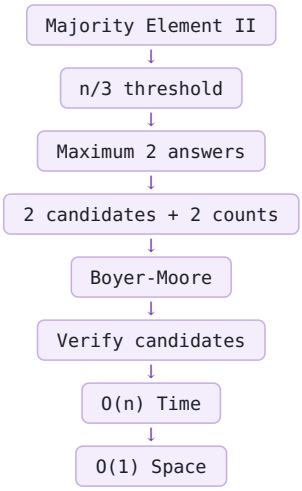
```java
import java.util.*;

class Solution {

    public static List<Integer> majorityElement(int[] nums) {

        int candidate1 = 0;
        int candidate2 = 0;

        int count1 = 0;
        int count2 = 0;

        // Find potential candidates
        for (int num : nums) {

            if (num == candidate1) {
                count1++;
            }

            else if (num == candidate2) {
                count2++;
            }

            else if (count1 == 0) {
                candidate1 = num;
                count1 = 1;
            }

            else if (count2 == 0) {
                candidate2 = num;
                count2 = 1;
            }

            else {
                count1--;
                count2--;
            }
        }

        // Verify candidates
        count1 = 0;
        count2 = 0;

        for (int num : nums) {

            if (num == candidate1) {
                count1++;
            }

            else if (num == candidate2) {
                count2++;
            }
        }

        List<Integer> ans = new ArrayList<>();

        if (count1 > nums.length / 3) {
            ans.add(candidate1);
        }

        if (count2 > nums.length / 3) {
            ans.add(candidate2);
        }

        return ans;
    }

    public static void main(String[] args) {

        int[] nums = {3, 2, 3};

        List<Integer> result = majorityElement(nums);

        System.out.println(result);
```

```
    }
}
```

## COMPLEXITY

Time: O(n) Space: O(1)

## CORE LOGIC




## COMMON MISTAKES

1. Sirf 1 candidate maintain karna. 2. Candidates ko directly answer maan lena. 3. >= n/3 use karna; correct is > n/3. 4. Final frequency verification bhoolna.

# Q23 Next Permutation

## INPUT

```
Integer array nums
```

## OUTPUT

Array ko next lexicographically greater permutation mein rearrange karna hai. Agar next greater permutation possible nahi hai, array ko ascending order mein arrange karo.

## EXAMPLE

Input: [1,2,3] Output: [1,3,2]

Input: [3,2,1] Output: [1,2,3]

## BRUTE FORCE

1. Saari possible permutations generate karo. 2. Current permutation se greater permutations find karo. 3. Smallest greater permutation choose karo.

Time: O(n!) Space: O(n!)

## OPTIMIZED APPROACH

3 Steps:

1. Pivot find karo. 2. Pivot se just greater element find karke swap karo. 3. Pivot ke right portion ko reverse karo.

STEP 1: FIND PIVOT

Right se traverse karo.

Condition: nums[i] < nums[i+1]

Rightmost aisa index pivot hoga.

Example: [1,3,5,4,2]

Right se: 5 > 4 3 < 5 → Pivot = 1

[1,3 | 5,4,2]

STEP 2: FIND NEXT GREATER ELEMENT

Pivot ke right side se right to left traverse karo.

First element jo: nums[i] > nums[pivot]

ho, usko pivot ke saath swap karo.

[1,3 | 5,4,2]

```
4 > 3 → YES
```

Swap: [1,4,5,3,2]

STEP 3: REVERSE SUFFIX

Pivot ke baad ka portion reverse karo.

[1,4 | 5,3,2]

Reverse: [1,4 | 2,3,5]

Final: [1,4,2,3,5]

### SPECIAL CASE

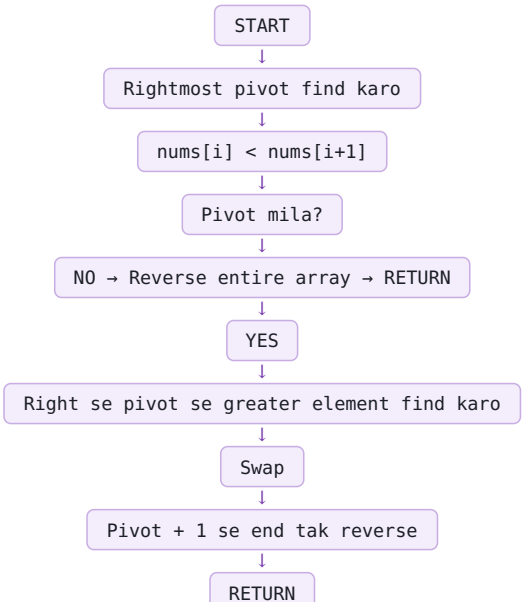
```
Agar pivot = -1:
```

[3,2,1]

Array already largest permutation hai.

Entire array reverse: [1,2,3]

## WORKFLOW



## DRY RUN

Input: [1,3,5,4,2]

```
Pivot = 1
```

[1,3,5,4,2]

```
Next greater than 3 = 4
```

Swap: [1,4,5,3,2]

Reverse suffix: [1,4,2,3,5]

Final Answer: [1,4,2,3,5]

## EDGE CASES

```
[1,2,3] → [1,3,2]
[3,2,1] → [1,2,3]
[1,1,5] → [1,5,1]
[1] → [1]
[1,5,1] → [5,1,1]
```

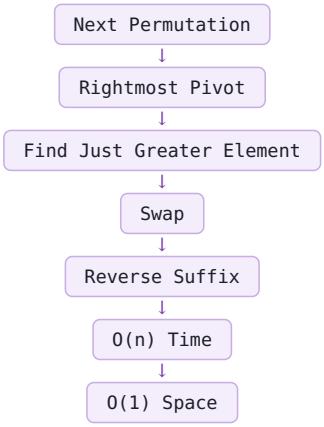
## COMPLETE JAVA CODE

```java
import java.util.*;

class Solution {

    public static void nextPermutation(int[] nums) {

        int n = nums.length;

        // Step 1: Find pivot
        int pivot = -1;

        for (int i = n - 2; i >= 0; i--) {

            if (nums[i] < nums[i + 1]) {
                pivot = i;
                break;
            }
        }

        // Step 2: No pivot means largest permutation
        if (pivot == -1) {
            reverse(nums, 0, n - 1);
            return;
        }

        // Step 3: Find next greater element
        for (int i = n - 1; i > pivot; i--) {

            if (nums[i] > nums[pivot]) {
                swap(nums, i, pivot);
                break;
            }
        }

        // Step 4: Reverse suffix
        reverse(nums, pivot + 1, n - 1);
    }

    public static void swap(int[] nums, int i, int j) {

        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }

    public static void reverse(int[] nums, int left, int right) {

        while (left < right) {

            swap(nums, left, right);

            left++;
            right--;
        }
    }

    public static void main(String[] args) {

        int[] nums = {1, 2, 3};

        nextPermutation(nums);

        System.out.println(Arrays.toString(nums));
    }
}
```

## COMPLEXITY

Time: O(n) Space: O(1)

## CORE LOGIC




### IMPORTANT

```
Java mein int[] ke liye direct Arrays.reverse() available nahi hai.
Isliye two-pointer se manually reverse karna padega.

Collections.reverse() List ke liye available hai,
lekin int[] ke liye nahi.
```

### COMMON MISTAKES

1. Pivot ko left se find karna. 2. Pivot ke just greater element ko choose na karna. 3. Swap ke baad suffix reverse na karna. 4. Pivot na mile to entire array reverse karna bhoolna. 5. Extra array use karna.

# Q24 First Missing Positive

## INPUT

```
Integer array nums
```

## OUTPUT

Smallest positive integer jo array mein missing hai.

## EXAMPLE

Input: [3,4,-1,1] Output: 2
Input: [1,2,0] Output: 3

## BRUTE FORCE

1. 1 se start karo. 2. Check karo number array mein present hai ya nahi. 3. Jo first positive number missing mile, wahi answer.
Time: O(n²) Space: O(1)

## OPTIMIZED APPROACH

Index Mapping / Cyclic Placement
Agar valid positive number x hai:

```
x → index x - 1

Example:
1 → index 0
2 → index 1
3 → index 2
4 → index 3

Sirf:
1 <= nums[i] <= n
```

wale numbers ko correct position par place karenge.
Negative, 0 aur n se bade numbers ko ignore karenge.

## MAIN LOGIC

Har index par:

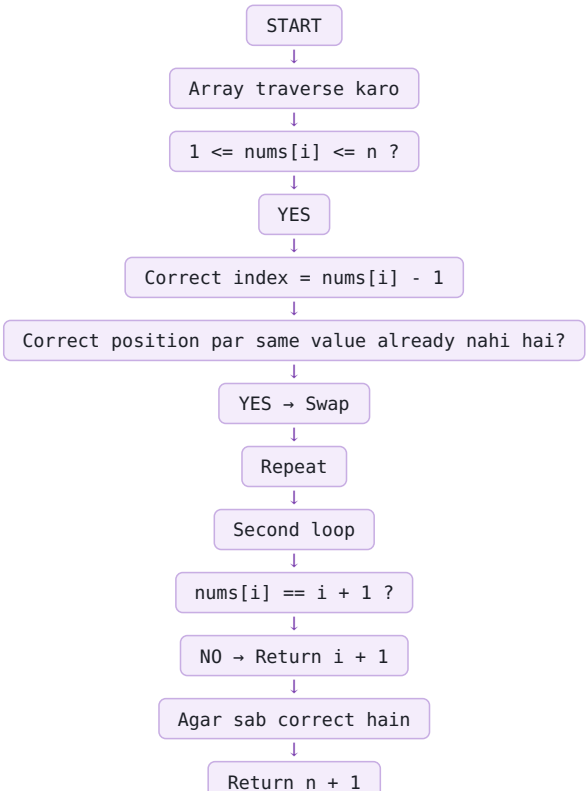
```
Agar:
nums[i] >= 1
AND
nums[i] <= n

to us number ko:
index = nums[i] - 1
```

par place karo.

```
Duplicate se bachne ke liye:
nums[nums[i] - 1] != nums[i]
```

condition bhi check karo.

## WORKFLOW



## DRY RUN

Input: [3,4,-1,1]

Valid values ko correct index par place karne ke baad:

[1,-1,3,4]

Check:

```
index 0:
nums[0] = 1
Expected = 1 → Correct

index 1:
nums[1] = -1
Expected = 2 → Incorrect

Answer = 2
```

## EDGE CASES

```
[1] → 2
[1,2,3] → 4
[3,4,-1,1] → 2
[1,2,0] → 3
[-1,-2,-3] → 1
[1,1] → 2
[2,2] → 1
```

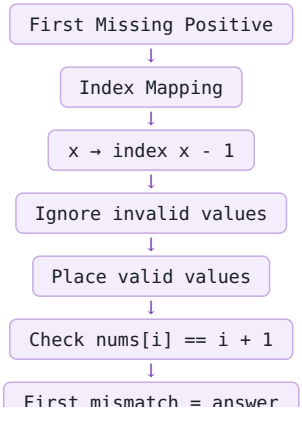
## COMPLETE JAVA CODE

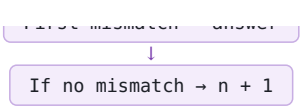
```
class Solution {

    public static int firstMissingPositive(int[] nums) {

        int n = nums.length;

        for (int i = 0; i < n; i++) {

            while (nums[i] >= 1 &&
                   nums[i] <= n &&
                   nums[nums[i] - 1] != nums[i]) {

                int correctIndex = nums[i] - 1;

                int temp = nums[i];
                nums[i] = nums[correctIndex];
                nums[correctIndex] = temp;
            }
        }

        for (int i = 0; i < n; i++) {

            if (nums[i] != i + 1) {
                return i + 1;
            }
        }

        return n + 1;
    }

    public static void main(String[] args) {

        int[] nums = {3,4,-1,1};

        int result = firstMissingPositive(nums);

        System.out.println(result);
    }
}
```

## COMPLEXITY

Time: O(n) Space: O(1)

## CORE LOGIC






**COMMON MISTAKES**

1. Negative numbers ko place karna. 2. 0 ko consider karna. 3. n se bade numbers ko place karna. 4. Duplicate values ki wajah se infinite loop create karna. 5. nums[i] != i + 1 check bhoolna. 6. Sab 1...n present hone par n + 1 return na karna.

# Q25 Merge Sorted Array

## INPUT

```
Two sorted integer arrays nums1 and nums2.
m = nums1 ke actual elements.
n = nums2 ke elements.
```

## OUTPUT

Dono arrays ko nums1 ke andar sorted order mein merge karna hai.

## EXAMPLE

```
nums1 = [1,2,3,0,0,0], m=3
nums2 = [2,5,6], n=3
```

Output: [1,2,2,3,5,6]

## BRUTE FORCE

1. nums2 ko nums1 ke empty space mein copy karo. 2. Pura nums1 sort karo.

Time: O((m+n) log(m+n)) Space: O(1) if in-place sorting.

## OPTIMIZED APPROACH

Two Pointer / In-place

Right se merge karenge because nums1 ke end mein empty space hai.

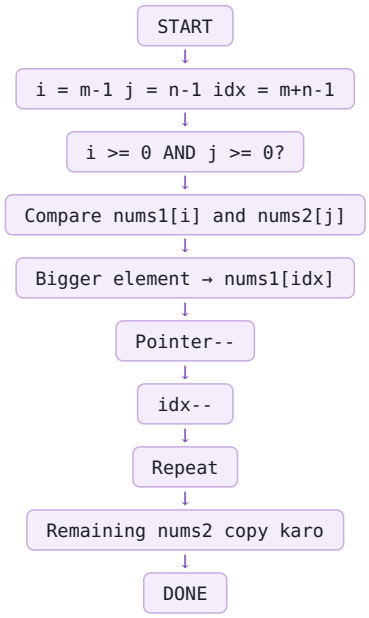
```
3 POINTERS:
i = m - 1
j = n - 1
idx = m + n - 1
```

## MAIN LOGIC

```
Agar nums1[i] >= nums2[j]:
    nums1[idx] = nums1[i]
    i--
Else:
    nums1[idx] = nums2[j]
    j--

Har case mein:
idx--

Agar nums2 ke elements bach gaye:
while (j >= 0)
    nums1[idx--] = nums2[j--]
```

nums1 ke remaining elements ko copy karne ki zarurat nahi hai.

## WORKFLOW




## DRY RUN

```
nums1 = [1,2,3,0,0,0]
nums2 = [2,5,6]

i=2 → 3
j=2 → 6
idx=5

6 → end
5 → end
3 → end
```

```
2 → end
```

Final: [1,2,2,3,5,6]

## EDGE CASES

```
[1], [] → [1]
[], [1] → [1]
[1,2,3,0,0,0], [4,5,6] → [1,2,3,4,5,6]
[4,5,6,0,0,0], [1,2,3] → [1,2,3,4,5,6]
```

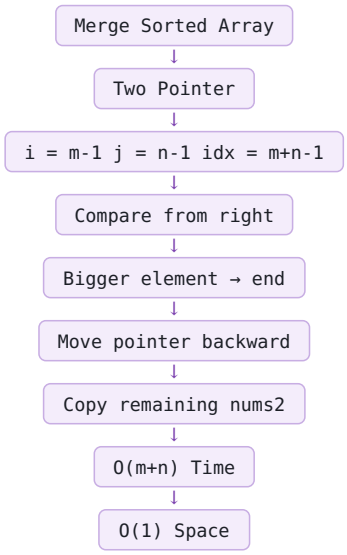
## COMPLETE JAVA CODE

```java
import java.util.*;

class Solution {

    public static void merge(int[] nums1, int m, int[] nums2, int n) {

        int idx = m + n - 1;
        int i = m - 1;
        int j = n - 1;

        while (i >= 0 && j >= 0) {

            if (nums1[i] >= nums2[j]) {
                nums1[idx--] = nums1[i--];
            } else {
                nums1[idx--] = nums2[j--];
            }
        }

        while (j >= 0) {
            nums1[idx--] = nums2[j--];
        }
    }

    public static void main(String[] args) {

        int[] nums1 = {1, 2, 3, 0, 0, 0};
        int m = 3;

        int[] nums2 = {2, 5, 6};
        int n = 3;

        merge(nums1, m, nums2, n);

        System.out.println(Arrays.toString(nums1));
    }
}
```

## COMPLEXITY

Time: O(m+n) Space: O(1)

## CORE LOGIC




## COMMON MISTAKES

1. Left se merge karna. 2. idx ko m+n-1 se start na karna. 3. i ko m-1 ki jagah n-1 karna. 4. nums1 ke zero values ko actual elements samajhna. 5. Remaining nums2 copy karna bhoolna. 6. Remaining nums1 ko copy karna; zaroorat nahi hai.

# Q26 Find All Numbers Disappeared In An Array

## INPUT

```
Integer array nums of size n.
Every value is between 1 and n.
```

## OUTPUT

1 se n ke beech ke saare missing numbers.

## EXAMPLE

Input: [4,3,2,7,8,2,3,1]

Output: [5,6]

## BRUTE FORCE

1. 1 se n tak har number check karo. 2. Check karo number array mein present hai ya nahi. 3. Missing number ko answer mein add karo.

Time: $O(n^2)$ Space: O(1)

## OPTIMIZED APPROACH

Index Marking / Sign Marking

Array ko hi marking ke liye use karenge.

```
For number x:
index = x - 1
```

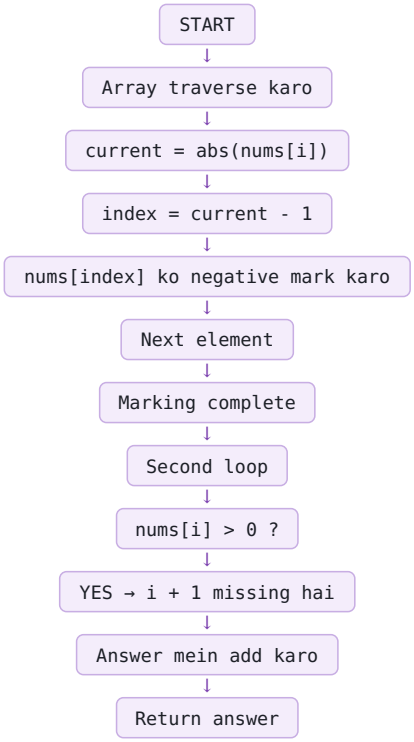
Present number ke corresponding index ko negative mark karenge.

```
Negative = Present
Positive = Missing
```

## MAIN LOGIC

```
int index = Math.abs(nums[i]) - 1;

if (nums[index] > 0) {
    nums[index] = -nums[index];
}

Math.abs() isliye use karte hain kyunki marking ke baad nums[i] negative ho sakta hai.
```

## WORKFLOW




## DRY RUN

Input: [4,3,2,7,8,2,3,1]

Marking ke baad: [-4,-3,-2,-7,8,2,-3,-1]

```
Positive positions:
index 4 → number 5
index 5 → number 6
```

Answer: [5,6]

## IMPORTANT

```
Agar nums[i] = 3:
```

```
index = 3 - 1 = 2
```

So index 2 ko mark karenge.

```
Final:
if (nums[i] > 0)
    ans.add(i + 1);
```

## EDGE CASES

```
[1] → []
[1,2,3] → []
[2,2] → [1]
[1,1] → [2]
[4,3,2,7,8,2,3,1] → [5,6]
```

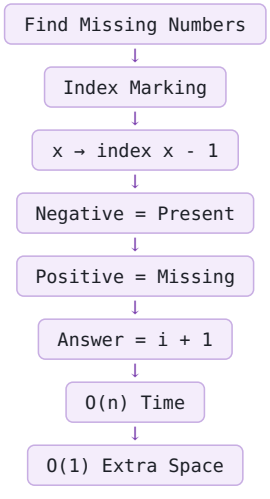
## COMPLETE JAVA CODE

```
import java.util.*;

class Solution {

    public static List<Integer> findDisappearedNumbers(int[] nums) {

        List<Integer> ans = new ArrayList<>();

        // Step 1: Mark present numbers
        for (int i = 0; i < nums.length; i++) {

            int index = Math.abs(nums[i]) - 1;

            if (nums[index] > 0) {
                nums[index] = -nums[index];
            }
        }

        // Step 2: Find unmarked indexes
        for (int i = 0; i < nums.length; i++) {

            if (nums[i] > 0) {
                ans.add(i + 1);
            }
        }

        return ans;
    }

    public static void main(String[] args) {

        int[] nums = {4,3,2,7,8,2,3,1};

        List<Integer> result = findDisappearedNumbers(nums);

        System.out.println(result);
    }
}
```

## COMPLEXITY

Time: O(n) Space: O(1) extra space

## CORE LOGIC




## COMMON MISTAKES

1. Math.abs() na lagana. 2. Already negative value ko positive kar dena. 3. i ki jagah i+1 add karna. 4. Separate boolean array banana jab O(1) space expected ho.

# Q27 Pascal's Triangle

## INPUT

```
Integer numRows.
```

## OUTPUT

First numRows rows of Pascal's Triangle.

## EXAMPLE

```
Input:
numRows = 5
```

Output:
[ [1], [1,1], [1,2,1], [1,3,3,1], [1,4,6,4,1] ]

## BASIC APPROACH

Har row ko previous row se build karenge.

```
First element = 1
Last element = 1
```

Middle element:
previousRow[j-1] + previousRow[j]

## EXAMPLE

Previous row:
[1,3,3,1]
Next row:
[1,4,6,4,1]
Because:

```
1
1 + 3 = 4
3 + 3 = 6
3 + 1 = 4
1
```

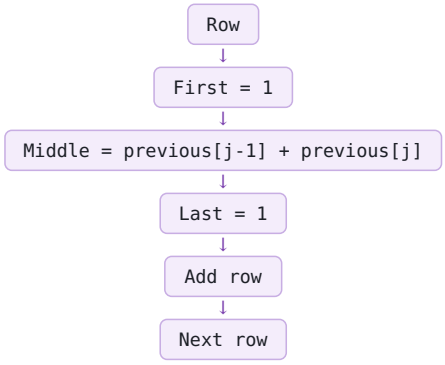
## COMPLETE JAVA CODE

```
import java.util.*;

class Solution {

    public static List<List<Integer>> generate(int numRows) {

        List<List<Integer>> triangle = new ArrayList<>();

        for (int i = 0; i < numRows; i++) {

            List<Integer> row = new ArrayList<>();

            for (int j = 0; j <= i; j++) {

                if (j == 0 || j == i) {

                    row.add(1);

                } else {

                    int value =
                        triangle.get(i - 1).get(j - 1)
                        +
                        triangle.get(i - 1).get(j);

                    row.add(value);
                }
            }

            triangle.add(row);
        }

        return triangle;
    }

    public static void main(String[] args) {

        System.out.println(generate(5));
    }
}
```

## COMPLEXITY

Time: $O(n^2)$
Space: $O(n^2)$ output

## CORE LOGIC




**COMMON MISTAKES**

1. First/last element ko calculate karna. 2. Previous row ke wrong indexes use karna. 3. j <= i condition bhoolna.

# Q28 Find Pivot Index

## INPUT

```
Integer array nums.
```

## OUTPUT

```
Woh index return karo jahan:

leftSum == rightSum
```

Pivot element left/right sum mein include nahi hota.

```
Multiple pivot indexes hon to leftmost return karo.

Agar koi pivot nahi hai:
-1 return karo.
```

## EXAMPLE

Input: [1,7,3,6,5,6]
Output: 3
Because:

```
Left:
1 + 7 + 3 = 11

Right:
5 + 6 = 11
```

## BRUTE FORCE

Har index ko pivot maan kar:
1. Left side ka sum calculate karo. 2. Right side ka sum calculate karo. 3. Dono equal hain to index return karo.
Time: $O(n^2)$
Space: O(1)

## OPTIMIZED APPROACH

Pehle complete array ka:
totalSum
calculate karo.
Then:

```
leftSum = 0
```

Right sum:

```
rightSum =
totalSum - leftSum - nums[i]
```

### PIVOT CONDITION

```
if (leftSum == rightSum)

    return i;
```

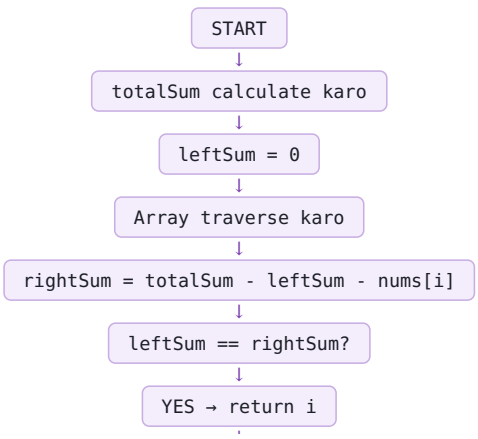
Agar pivot nahi mila:

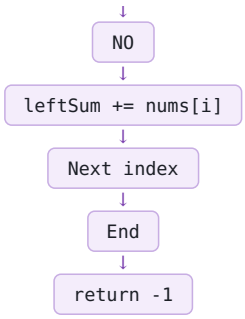
```
leftSum += nums[i]
```

### IMPORTANT

leftSum ko condition check ke BAAD update karna hai.
Agar pehle update kiya, to pivot element left sum mein include ho jayega.

## WORKFLOW






## DRY RUN

Input:

[1,7,3,6,5,6]

```
totalSum = 28
leftSum = 0

i = 0:
rightSum = 28 - 0 - 1 = 27
leftSum = 1

i = 1:
rightSum = 28 - 1 - 7 = 20
leftSum = 8

i = 2:
rightSum = 28 - 8 - 3 = 17
leftSum = 11

i = 3:
rightSum = 28 - 11 - 6 = 11

11 == 11

Answer = 3
```

## EDGE CASES

```
[1,7,3,6,5,6] → 3
[1,2,3] → -1
[2,1,-1] → 0
[1] → 0
[0,0,0] → 0
```

## COMPLETE JAVA CODE

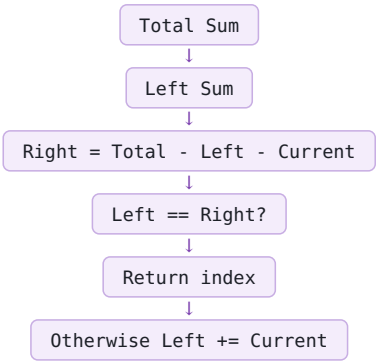
```
class Solution {

    public static int pivotIndex(int[] nums) {

        int totalSum = 0;

        for (int i = 0; i < nums.length; i++) {
            totalSum += nums[i];
        }

        int leftSum = 0;

        for (int i = 0; i < nums.length; i++) {

            int rightSum =
                totalSum - leftSum - nums[i];

            if (leftSum == rightSum) {
                return i;
            }

            leftSum += nums[i];
        }

        return -1;
    }

    public static void main(String[] args) {

        int[] nums = {1,7,3,6,5,6};

        int result = pivotIndex(nums);

        System.out.println(result);
    }
}
```

## COMPLEXITY

Time: O(n)

Space: O(1)

## CORE LOGIC




### COMMON MISTAKES

1. Pivot element ko sum mein include karna. 2. nums[i] return karna instead of i. 3. leftSum ko condition se pehle update karna. 4. Right sum formula galat likhna. 5. No pivot par -1 return bhoolna.

# Q29 Difference Of Two Arrays

## INPUT

Two integer arrays:
nums1 nums2

## OUTPUT

Two lists:
1. nums1 ke elements jo nums2 mein present nahi hain. 2. nums2 ke elements jo nums1 mein present nahi hain.
Duplicate values answer mein sirf ek baar.

## EXAMPLE

Input:

```
nums1 = [1,2,3]
nums2 = [2,4,6]
```

Output:
[[1,3],[4,6]]

## BRUTE FORCE

```
1. nums1 ke har element ko nums2 mein search karo.
2. Jo present nahi hai → first list mein add karo.
3. nums2 ke har element ko nums1 mein search karo.
4. Jo present nahi hai → second list mein add karo.
```

Time: O(n*m)
Agar dono arrays ki size n ho:
$O(n^2)$

## OPTIMIZED APPROACH

```
HashSet.

nums1 → set1
nums2 → set2

HashSet duplicates automatically remove karta hai.
```

First list:
set1 ke elements traverse karo.
Agar:

```
!set2.contains(num)
```

to:

```
list1.add(num)
```

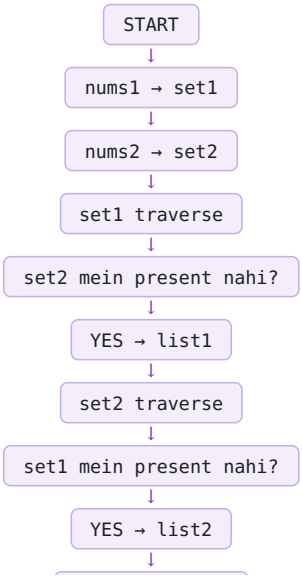
Second list:
set2 ke elements traverse karo.
Agar:

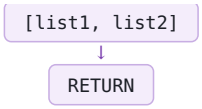
```
!set1.contains(num)
```

to:

```
list2.add(num)
```

## WORKFLOW






## DUPLICATE EXAMPLE

```
nums1 = [1,2,3,3]
nums2 = [2,4,6]

set1 = {1,2,3}
```

Answer:
[[1,3],[4,6]]

## COMPLETE JAVA CODE

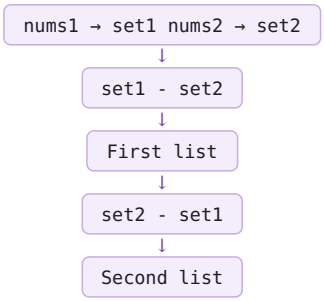
```
import java.util.*;

class Solution {

    public static List<List<Integer>> findDifference(
            int[] nums1, int[] nums2) {

        Set<Integer> set1 = new HashSet<>();
        Set<Integer> set2 = new HashSet<>();

        for (int num : nums1) {
            set1.add(num);
        }

        for (int num : nums2) {
            set2.add(num);
        }

        List<Integer> list1 = new ArrayList<>();
        List<Integer> list2 = new ArrayList<>();

        for (int num : set1) {

            if (!set2.contains(num)) {
                list1.add(num);
            }
        }

        for (int num : set2) {

            if (!set1.contains(num)) {
                list2.add(num);
            }
        }

        List<List<Integer>> ans = new ArrayList<>();

        ans.add(list1);
        ans.add(list2);

        return ans;
    }

    public static void main(String[] args) {

        int[] nums1 = {1,2,3};
        int[] nums2 = {2,4,6};

        List<List<Integer>> result =
            findDifference(nums1, nums2);

        System.out.println(result);
    }
}
```

## COMPLEXITY

Time: O(n + m) average

Space: O(n + m)

## CORE LOGIC




## COMMON MISTAKES

1. Array mein contains() use karna. 2. Duplicate manually count karna. 3. Sirf one-way difference nikalna. 4. set1/set2 direction confuse karna.

**REMAINING QUESTIONS**

```
28 → Pascal's Triangle
29 → Find Pivot Index
30 → Difference of Two Arrays
```